

HK0411Bv1RV32

Specification Document v1.0

Haresh Krishna GS

June 14, 2026

Contents

I	ARCHITECTURAL SPECIFICATION	5
1	Introduction	5
2	Design Parameters	6
2.1	Processor Configuration	6
2.2	Pipeline Organization	6
2.3	Supported Instruction Categories	6
2.4	Design Priorities	7
3	Supported Instruction Set	8
3.1	Register-Register Arithmetic Instructions	8
3.2	Immediate Arithmetic Instructions	8
3.3	Load Instructions	8
3.4	Store Instructions	8
3.5	Branch Instructions	9
3.6	Jump Instructions	9
3.7	Upper Immediate Instructions	9
3.8	Unsupported RV32I Instructions	9
4	Instruction Formats	10
4.1	R-Type Format	10
4.2	I-Type Format	10
4.3	S-Type Format	10
4.4	B-Type Format	11
4.5	U-Type Format	11
4.6	J-Type Format	12
5	Supported Instruction Encoding Specification	13
5.1	Supported Instructions	13
5.2	Instruction Format Usage	13
5.3	R-Type Instruction Encodings	13
5.4	I-Type Instruction Encodings	14
5.5	Load Instruction Encodings	14
5.6	Store Instruction Encodings	14
5.7	Branch Instruction Encodings	14

5.8	Jump Instruction Encodings	14
5.9	Upper Immediate Instruction Encodings	14
II	MICROARCHITECTURAL SPECIFICATION	15
6	Pipeline Overview	15
6.1	Pipeline Stages	15
6.1.1	Instruction Fetch (IF)	15
6.1.2	Instruction Decode (ID)	15
6.1.3	Execute (EX)	16
6.1.4	Memory Access (MEM)	16
6.1.5	Write Back (WB)	16
6.2	Pipeline Operation Example	17
6.3	Pipeline Performance	17
7	Datapath Architecture	18
7.1	Top-Level Datapath Overview	18
7.2	Instruction Fetch (IF) Datapath	19
7.3	Instruction Decode (ID) Datapath	19
7.4	Execute (EX) Datapath	20
7.5	Memory Access (MEM) Datapath	20
7.6	Write Back (WB) Datapath	21
7.7	Datapath Design Decisions	21
8	Control Path Overview	22
8.1	Control Path Organization	22
8.2	Main Control Unit	23
8.3	ALU Control Unit	23
8.4	Branch Control Unit	24
8.5	Forwarding Unit	24
8.6	Hazard Detection Unit	24
8.7	Pipeline Flush Controller	25
8.8	Control Signal Summary	25
9	Immediate Generator Specification	26
9.1	Supported Immediate Types	26
9.2	I-Type Immediate	26
9.3	S-Type Immediate	27
9.4	B-Type Immediate	27
9.5	U-Type Immediate	27
9.6	J-Type Immediate	28
9.7	Immediate Type Selection	28
9.8	Module Interface	28
9.9	Design Requirements	28
10	Control Signal Specification	29
10.1	Control Signal Summary	29
10.2	RegWrite	29
10.3	MemRead	29
10.4	MemWrite	29
10.5	Branch	29

10.6	Jump	30
10.7	ALUSrc	30
10.8	WBSelect	30
10.9	ALUOp	30
10.10	Instruction Control Signal Truth Table	31
11	ALU Operation Encoding	32
11.1	ALU Control Signal Specification	32
11.2	ALU Control Encoding	32
11.3	Supported ALU Operations	32
11.4	ALU Control Generation Rules	33
11.4.1	Load and Store Instructions	33
11.4.2	Branch Instructions	33
11.4.3	Jump Instructions	33
11.4.4	Upper Immediate Instructions	33
11.5	Complete Instruction-to-ALU Control Mapping	34
11.6	ALU Zero Flag	34
12	Pipeline Register Specification	35
12.1	IF/ID Pipeline Register	35
12.1.1	Stored Signals	35
12.1.2	Functional Requirements	35
12.2	ID/EX Pipeline Register	35
12.2.1	Datapath Signals	36
12.2.2	Control Signals	36
12.2.3	Functional Requirements	36
12.3	EX/MEM Pipeline Register	36
12.3.1	Datapath Signals	36
12.3.2	Control Signals	37
12.3.3	Functional Requirements	37
12.4	MEM/WB Pipeline Register	37
12.4.1	Datapath Signals	37
12.4.2	Control Signals	37
12.4.3	Functional Requirements	37
12.5	Writeback Source Encoding	37
12.6	Pipeline Register Summary	38
13	Hazard Management Overview	39
13.1	Structural Hazards	39
13.2	Data Hazards	39
13.3	Forwarding Mechanism	39
13.4	Load-Use Hazards	39
13.5	Control Hazards	40
13.6	Hazard Management Summary	40
14	Branch Prediction Strategy	41
14.1	Prediction Policy	41
14.2	Branch Resolution	41
14.3	Misprediction Recovery	41
14.4	Future Enhancements	41

15 Memory Architecture	42
15.1 Instruction Memory	42
15.2 Data Memory	42
15.3 Address Space	42
15.4 Memory Alignment	43
 III IMPLEMENTATION AND VERIFICATION SPECIFICATION	 44
16 RTL Module Specification	44
16.1 pc.sv	44
16.2 instr_mem.sv	44
16.3 data_mem.sv	44
16.4 regfile.sv	44
16.5 imm_gen.sv	44
16.6 alu.sv	44
16.7 alu_control.sv	44
16.8 control_unit.sv	44
16.9 forwarding_unit.sv	44
16.10hazard_unit.sv	44
16.11if_id.sv	44
16.12id_ex.sv	44
16.13ex_mem.sv	44
16.14mem_wb.sv	44
16.15cpu_top.sv	44
 17 Verification Requirements	 45
17.1 Verification Environment	45
17.2 Directed Testing	45
17.3 Hazard Verification	45
17.4 Functional Coverage	45
17.5 Assertion-Based Verification	46
17.6 Verification Completion Criteria	46
 18 Verification Plan	 46
18.1 Instruction Coverage Matrix	46
18.2 Hazard Coverage Matrix	46
18.3 Functional Coverage Plan	46
18.4 Assertion Plan	46

Part I

ARCHITECTURAL SPECIFICATION

1 Introduction

This document defines the Architectural Specification, Microarchitectural Specification and Implementation Specification of the HK0411Bv1RV32 processor. The HK0411Bv1RV32 is a 32-bit RISC-V processor implementing a subset of the RV32I Instruction Set Architecture (ISA). The processor employs a classic five-stage in-order pipeline consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB) stages.

The purpose of this document is to provide a detailed description of the internal organization, datapath structure, control path implementation, pipeline operation, hazard management mechanisms, and verification requirements of the processor.

This specification serves as the primary reference for:

- RTL Design
- Functional Verification
- Architecture Validation
- Performance Analysis
- Documentation and Maintenance

The processor is intended as an educational and professional RTL design project focused on demonstrating CPU microarchitecture concepts including:

- Instruction decoding
- Pipeline execution
- Data forwarding
- Hazard detection
- Branch handling
- Functional verification
- Assertion-based verification

The processor adopts a Harvard Architecture with separate instruction and data memories in order to eliminate structural hazards associated with memory accesses.

Branch instructions are resolved in the Execute stage. The processor implements a static branch prediction strategy that assumes all branches are not taken. Mispredicted branches are recovered through pipeline flushing and program counter redirection.

The design philosophy of HK0411Bv1RV32 prioritizes correctness, clean RTL implementation, and verification quality over advanced performance optimizations.

2 Design Parameters

2.1 Processor Configuration

Parameter	Value
Processor Name	HK0411Bv1RV32
ISA	RV32I Subset
Architecture Type	Harvard Architecture
Pipeline Organization	In-Order Pipeline
Pipeline Depth	5 Stages
Instruction Width	32 Bits
Data Width (XLEN)	32 Bits
Address Width	32 Bits
Register Count	32
Register Width	32 Bits
Register File Read Ports	2
Register File Write Ports	1
x0 Register	Hardwired to Zero
Issue Width	Single Issue
Retirement Width	Single Instruction
Instruction Memory	Dedicated
Data Memory	Dedicated
Branch Resolution Stage	Execute (EX)
Branch Prediction Policy	Static Not Taken
Forwarding Support	Enabled
Load-Use Hazard Detection	Enabled
Pipeline Flush Support	Enabled
Reset Type	Active High Synchronous

2.2 Pipeline Organization

The processor implements the following five-stage pipeline:

$$\text{IF} \rightarrow \text{ID} \rightarrow \text{EX} \rightarrow \text{MEM} \rightarrow \text{WB}$$

where:

- IF : Instruction Fetch
- ID : Instruction Decode and Register Read
- EX : Execute and Branch Resolution
- MEM : Data Memory Access
- WB : Register Write Back

2.3 Supported Instruction Categories

The processor supports the following RV32I instruction categories:

- Register-Register Arithmetic Instructions
- Immediate Arithmetic Instructions

- Load Instructions
- Store Instructions
- Branch Instructions
- Jump Instructions
- Upper Immediate Instructions

The total supported instruction count is twenty-five instructions.

2.4 Design Priorities

The implementation priorities of HK0411Bv1RV32 are:

1. Correct RV32I Functional Behavior
2. Hazard Detection and Recovery
3. Data Forwarding
4. Branch Flush Recovery
5. Self-Checking Verification Environment
6. Functional Coverage
7. Assertion-Based Verification
8. Static Branch Prediction

Dynamic branch prediction, cache hierarchy support, interrupts, exceptions, and multiplication/division extensions are intentionally excluded from Version 1 of the processor.

3 Supported Instruction Set

The HK0411Bv1RV32 processor implements a subset of the RV32I base integer instruction set architecture.

The objective of the supported instruction set is to provide sufficient functionality for arithmetic computation, logical operations, memory access, conditional branching, and program flow control while maintaining a manageable implementation complexity for Version 1 of the processor.

The processor supports a total of twenty-five instructions.

3.1 Register-Register Arithmetic Instructions

The following R-Type instructions are supported:

Instruction	Description
ADD	Integer Addition
SUB	Integer Subtraction
SLL	Logical Left Shift
SLT	Signed Set Less Than
SLTU	Unsigned Set Less Than
XOR	Bitwise Exclusive OR
SRL	Logical Right Shift
SRA	Arithmetic Right Shift
OR	Bitwise OR
AND	Bitwise AND

3.2 Immediate Arithmetic Instructions

The following I-Type arithmetic instructions are supported:

Instruction	Description
ADDI	Add Immediate
SLTI	Signed Set Less Than Immediate
SLTIU	Unsigned Set Less Than Immediate
XORI	Exclusive OR Immediate
ORI	OR Immediate
ANDI	AND Immediate
SLLI	Logical Left Shift Immediate
SRLI	Logical Right Shift Immediate
SRAI	Arithmetic Right Shift Immediate

3.3 Load Instructions

The processor supports the following load instruction:

Instruction	Description
LW	Load 32-bit Word From Memory

3.4 Store Instructions

The processor supports the following store instruction:

Instruction	Description
SW	Store 32-bit Word To Memory

3.5 Branch Instructions

The processor supports the following conditional branch instructions:

Instruction	Description
BEQ	Branch If Equal
BNE	Branch If Not Equal

3.6 Jump Instructions

The processor supports the following unconditional jump instruction:

Instruction	Description
JAL	Jump And Link

3.7 Upper Immediate Instructions

The processor supports the following upper-immediate instruction:

Instruction	Description
LUI	Load Upper Immediate

3.8 Unsupported RV32I Instructions

The following RV32I instructions are intentionally excluded from Version 1 of the HK0411Bv1RV32 processor.

Instruction	Description
LB	Load Byte
LH	Load Halfword
LBU	Load Byte Unsigned
LHU	Load Halfword Unsigned
SB	Store Byte
SH	Store Halfword
BLT	Branch if Less Than
BGE	Branch if Greater Than or Equal
BLTU	Branch if Less Than Unsigned
BGEU	Branch if Greater Than or Equal Unsigned
JALR	Jump and Link Register
AUIPC	Add Upper Immediate to Program Counter
FENCE	Memory Ordering Fence
FENCE.I	Instruction Stream Synchronization Fence
ECALL	Environment Call
EBREAK	Environment Breakpoint

The above instructions are not required for the baseline implementation and may be considered for future versions of the processor.

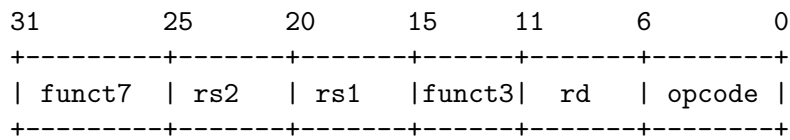
4 Instruction Formats

The HK0411Bv1RV32 processor supports six RV32I instruction formats:

- R-Type
- I-Type
- S-Type
- B-Type
- U-Type
- J-Type

4.1 R-Type Format

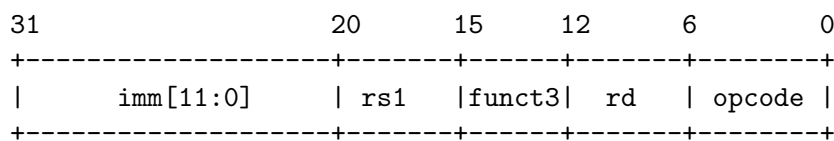
Used for register-register arithmetic and logical operations.



Field	Description
opcode	Instruction opcode
funct3	Function field
funct7	Function field
rs1	Source Register 1
rs2	Source Register 2
rd	Destination Register

4.2 I-Type Format

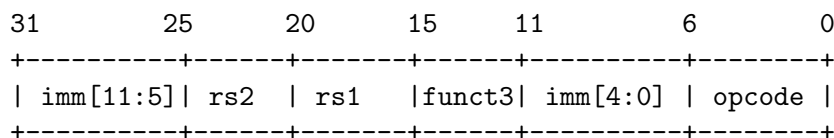
Used for immediate arithmetic operations and loads.



Field	Description
opcode	Instruction opcode
funct3	Function field
rs1	Source Register
rd	Destination Register
imm[11:0]	Sign Extended Immediate

4.3 S-Type Format

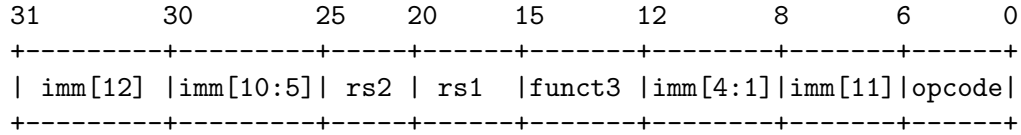
Used for store instructions.



Field	Description
opcode	Instruction opcode
funct3	Function field
rs1	Base Address Register
rs2	Store Data Register
imm	Store Offset

4.4 B-Type Format

Used for conditional branch instructions.



Field	Description
opcode	Instruction opcode
funct3	Branch type selector
rs1	Source Register 1
rs2	Source Register 2
imm[12]	Branch Immediate Bit 12
imm[10:5]	Branch Immediate Bits 10:5
imm[4:1]	Branch Immediate Bits 4:1
imm[11]	Branch Immediate Bit 11

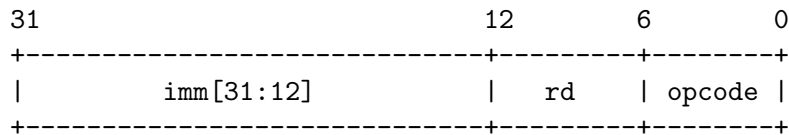
The branch offset shall be reconstructed as:

```
branch_imm = { {19{instr[31]}}},
              instr[31],
              instr[7],
              instr[30:25],
              instr[11:8],
              1'b0 };
```

The reconstructed immediate is sign-extended to XLEN bits.

4.5 U-Type Format

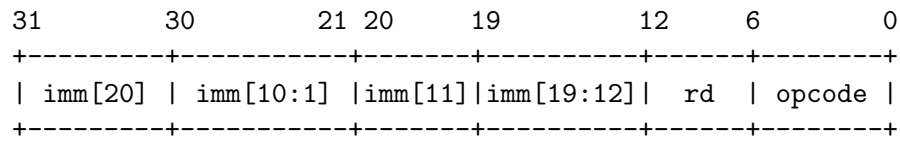
Used for upper immediate instructions.



Field	Description
opcode	Instruction opcode
rd	Destination Register
imm[31:12]	Upper Immediate

4.6 J-Type Format

Used for unconditional jump instructions.



Field	Description
opcode	Instruction opcode
rd	Destination Register
imm[20]	Jump Immediate Bit 20
imm[10:1]	Jump Immediate Bits 10:1
imm[11]	Jump Immediate Bit 11
imm[19:12]	Jump Immediate Bits 19:12

The jump offset shall be reconstructed as:

```

jal_imm = { {11{instr[31]}},
            instr[31],
            instr[19:12],
            instr[20],
            instr[30:21],
            1'b0 };

```

The reconstructed immediate is sign-extended to XLEN bits.

5 Supported Instruction Encoding Specification

The HK0411Bv1RV32 processor implements a subset of the RV32I instruction set architecture. The decoder shall recognize the instruction encodings defined in this section.

5.1 Supported Instructions

Category	Instructions
R-Type	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
I-Type	ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI
Load	LW
Store	SW
Branch	BEQ, BNE
Jump	JAL
Upper Immediate	LUI

The processor supports a total of 25 instructions.

5.2 Instruction Format Usage

Format	Instructions
R-Type	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
I-Type	ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI, LW
S-Type	SW
B-Type	BEQ, BNE
U-Type	LUI
J-Type	JAL

5.3 R-Type Instruction Encodings

Instruction	Opcode	Funct3	Funct7
ADD	0110011	000	0000000
SUB	0110011	000	0100000
SLL	0110011	001	0000000
SLT	0110011	010	0000000
SLTU	0110011	011	0000000
XOR	0110011	100	0000000
SRL	0110011	101	0000000
SRA	0110011	101	0100000
OR	0110011	110	0000000
AND	0110011	111	0000000

5.4 I-Type Instruction Encodings

Instruction	Opcode	Funct3	Funct7
ADDI	0010011	000	—
SLTI	0010011	010	—
SLTIU	0010011	011	—
XORI	0010011	100	—
ORI	0010011	110	—
ANDI	0010011	111	—
SLLI	0010011	001	0000000
SRLI	0010011	101	0000000
SRAI	0010011	101	0100000

5.5 Load Instruction Encodings

Instruction	Opcode	Funct3
LW	0000011	010

5.6 Store Instruction Encodings

Instruction	Opcode	Funct3
SW	0100011	010

5.7 Branch Instruction Encodings

Instruction	Opcode	Funct3
BEQ	1100011	000
BNE	1100011	001

5.8 Jump Instruction Encodings

Instruction	Opcode
JAL	1101111

5.9 Upper Immediate Instruction Encodings

Instruction	Opcode
LUI	0110111

Part II

MICROARCHITECTURAL SPECIFICATION

6 Pipeline Overview

The HK0411Bv1RV32 processor implements a classic five-stage in-order pipeline. The pipeline architecture improves instruction throughput by allowing multiple instructions to execute concurrently in different stages of the processor.

The implemented pipeline stages are:

$$\text{IF} \rightarrow \text{ID} \rightarrow \text{EX} \rightarrow \text{MEM} \rightarrow \text{WB}$$

Under ideal operating conditions, the processor is capable of completing one instruction every clock cycle after the pipeline is filled. The processor is a single-issue, in-order execution machine and does not support out-of-order execution or speculative execution.

6.1 Pipeline Stages

The functionality of each pipeline stage is described below.

6.1.1 Instruction Fetch (IF)

Responsibilities:

- Read the Program Counter (PC)
- Fetch the instruction from Instruction Memory
- Compute the sequential next address $\text{PC} + 4$
- Select the next PC value based on branch or jump decisions

Stage Outputs:

Signal	Description
Instruction	32-bit fetched instruction word
Current PC	Address of the fetched instruction
$\text{PC} + 4$	Sequential next instruction address

6.1.2 Instruction Decode (ID)

Responsibilities:

- Decode instruction fields (opcode, funct3, funct7, rs1, rs2, rd)
- Generate all required control signals
- Read source operands from the Register File
- Generate and sign-extend immediate values

Stage Outputs:

Signal	Description
RS1 Data	Value read from source register 1
RS2 Data	Value read from source register 2
Immediate Value	Sign-extended immediate operand
RD Address	Destination register number
Control Signals	Decoded control signal bundle

6.1.3 Execute (EX)

Responsibilities:

- Execute ALU operations on selected operands
- Calculate the branch target address
- Evaluate the branch condition
- Calculate effective addresses for load and store instructions

Stage Outputs:

Signal	Description
ALU Result	Result of the arithmetic/logic unit
Branch Taken	Branch condition evaluation result
Branch Target	Computed branch or jump target address

Branch instructions are fully resolved in this stage. A mispredicted branch causes a pipeline flush and program counter redirection.

6.1.4 Memory Access (MEM)

Responsibilities:

- Read from data memory for load instructions
- Write to data memory for store instructions

Stage Outputs:

Signal	Description
Memory Read Data	Data word read from data memory
ALU Result Passthrough	ALU result forwarded to Write Back

6.1.5 Write Back (WB)

Responsibilities:

- Select the appropriate writeback data source
- Write the result to the destination register in the Register File

Writeback Sources:

Source	Applicable Instructions
ALU Result	R-Type and I-Type arithmetic
Data Memory Output	Load instructions (LW)
PC + 4	Jump and link (JAL)
Upper Immediate	Load upper immediate (LUI)

6.2 Pipeline Operation Example

The following example illustrates the concurrent pipeline execution of three independent instructions:

I1: ADD x1, x2, x3
I2: SUB x4, x5, x6
I3: AND x7, x8, x9

Pipeline Execution Diagram:

Cycle	I1 (ADD)	I2 (SUB)	I3 (AND)	I4	I5
1	IF	—	—	—	—
2	ID	IF	—	—	—
3	EX	ID	IF	—	—
4	MEM	EX	ID	—	—
5	WB	MEM	EX	—	—
6	—	WB	MEM	—	—
7	—	—	WB	—	—

After the pipeline is fully filled, one instruction completes execution and retires every clock cycle under ideal conditions.

6.3 Pipeline Performance

Ideal Conditions:

Metric	Value
Cycles Per Instruction (CPI)	1
Retirement Rate	1 instruction per cycle
Maximum Throughput	1 instruction per clock cycle

CPI Degradation Causes:

Hazard Type	Effect on Pipeline
Data Hazard	Resolved by forwarding where possible
Load-Use Hazard	Inserts one pipeline stall cycle
Control Hazard	Resolved by static not-taken prediction
Branch Misprediction	Causes pipeline flush and redirect

7 Datapath Architecture

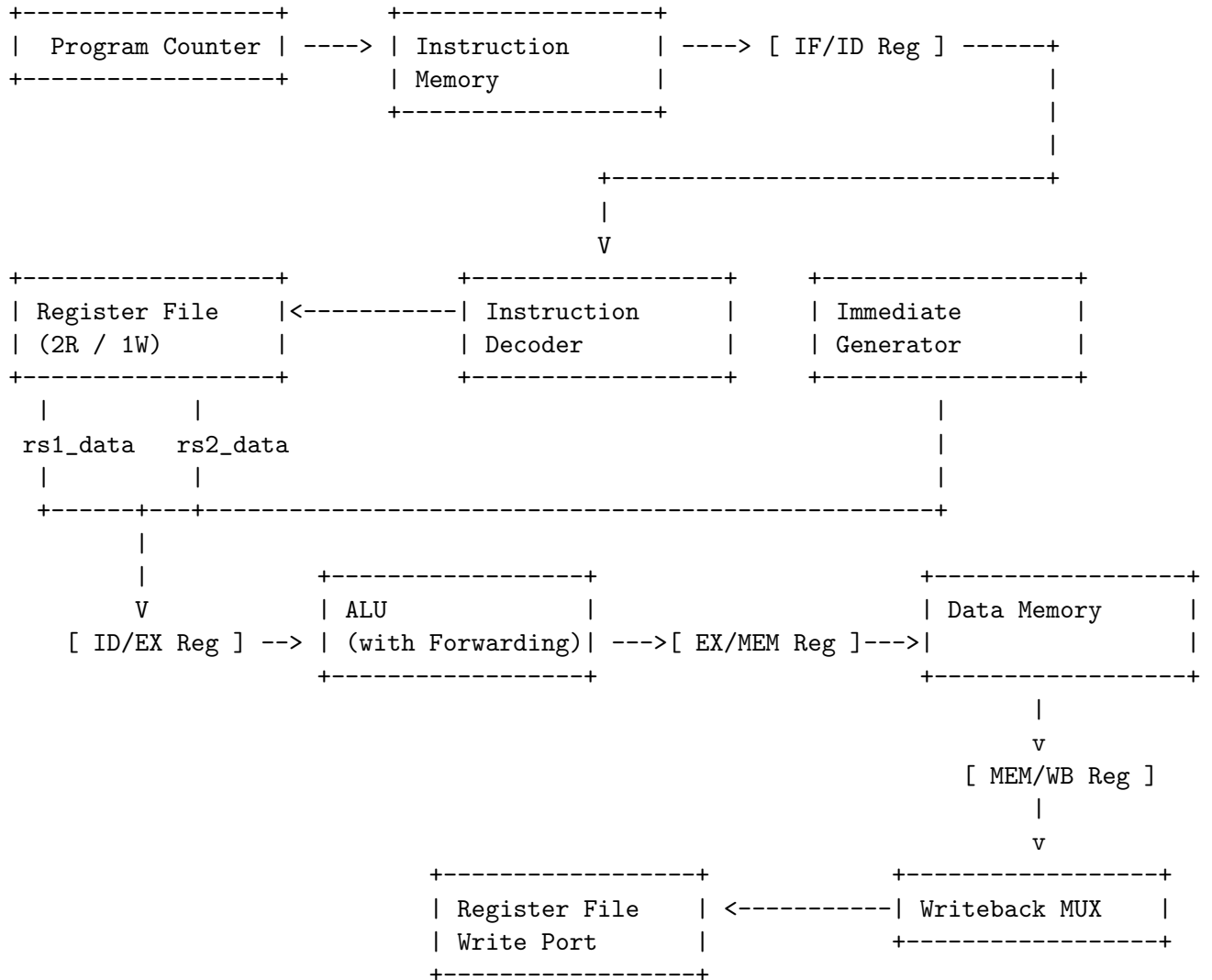
The HK0411Bv1RV32 processor implements a five-stage pipelined datapath. Each stage performs a well-defined set of operations and communicates with adjacent stages exclusively through dedicated pipeline registers.

The datapath is partitioned into the following stages:

Stage	Name
1	Instruction Fetch (IF)
2	Instruction Decode (ID)
3	Execute (EX)
4	Memory Access (MEM)
5	Write Back (WB)

Pipeline stages are separated by dedicated pipeline registers: IF/ID, ID/EX, EX/MEM, and MEM/WB.

7.1 Top-Level Datapath Overview



7.2 Instruction Fetch (IF) Datapath

The IF stage is responsible for fetching the next instruction from Instruction Memory and computing the sequential next PC value.

Stage Components:

Component	Function
Program Counter	Holds the address of the current instruction
PC + 4 Adder	Computes the sequential next instruction address
Next PC Multiplexer	Selects the next PC value
Instruction Memory	Provides the 32-bit instruction word

Next PC Multiplexer Select Sources:

Source	Condition
PC + 4	Default sequential fetch
Branch Target	Branch instruction taken
Jump Target	JAL instruction executed

Stage Interface:

Signal	Direction	Description
branch_target	Input	Target address from EX stage
branch_taken	Input	Branch decision from EX stage
instruction	Output	Fetches 32-bit instruction word
pc	Output	Current program counter value
pc_plus4	Output	PC + 4 sequential address

7.3 Instruction Decode (ID) Datapath

The ID stage decodes the fetched instruction, reads the register file, and generates all required control signals and immediate values.

Stage Components:

Component	Function
Instruction Decoder	Extracts opcode, funct3, funct7, rs1, rs2, rd
Main Control Unit	Generates all pipeline control signals
Register File	Provides two 32-bit source operand values
Immediate Generator	Constructs and sign-extends immediate values

Register File Specification:

Parameter	Value
Number of Registers	32
Register Width	32 bits
Read Ports	2
Write Ports	1
x0 Behaviour	Hardwired to zero

Stage Interface:

Signal	Direction	Description
rs1_data	Output	Value read from source register 1
rs2_data	Output	Value read from source register 2
immediate	Output	Sign-extended immediate value
rd	Output	Destination register address
control_signals	Output	Decoded control signal bundle

7.4 Execute (EX) Datapath

The EX stage performs arithmetic and logical operations, evaluates branch conditions, and computes memory access addresses.

Stage Components:

Component	Function
Forwarding Multiplexer A	Selects operand A with forwarding
Forwarding Multiplexer B	Selects operand B with forwarding
ALU Operand Multiplexer	Selects between register and immediate
ALU	Executes arithmetic and logic operations
Branch Comparator	Evaluates branch condition
Branch Target Adder	Computes branch target address

ALU Supported Operations:

Operation	Description
ADD	Integer addition
SUB	Integer subtraction
SLL	Logical left shift
SLT	Signed set less than
SLTU	Unsigned set less than
XOR	Bitwise exclusive OR
SRL	Logical right shift
SRA	Arithmetic right shift
OR	Bitwise OR
AND	Bitwise AND

Operand Source Selection:

Operand	Sources
Operand A	Register File, EX/MEM Forward, MEM/WB Forward
Operand B	Register File, Immediate, EX/MEM Forward, MEM/WB Forward

Branch Target Computation:

$\text{branch_target} = \text{pc} + \text{branch_imm}$

Supported Branch Conditions:

Instruction	Condition
BEQ	Branch if $\text{rs1} == \text{rs2}$
BNE	Branch if $\text{rs1} != \text{rs2}$

7.5 Memory Access (MEM) Datapath

The MEM stage performs reads and writes to the data memory using the ALU result as the effective memory address.

Stage Components:

Component	Function
Data Memory	Performs load and store word operations

Supported Memory Operations:

Instruction	Operation	Description
LW	Read	Load 32-bit word from memory
SW	Write	Store 32-bit word to memory

Address and Data Sources:

```
memory_address = alu_result
store_data     = rs2_data
```

7.6 Write Back (WB) Datapath

The WB stage selects the appropriate result and writes it back to the destination register in the Register File.

Stage Components:

Component	Function
Writeback Multiplexer	Selects the register writeback data source

Writeback Source Selection:

Source	Applicable Instructions
ALU Result	R-Type and I-Type arithmetic
Memory Read Data	Load instructions (LW)
PC + 4	Jump and link (JAL)
Upper Immediate Value	Load upper immediate (LUI)

7.7 Datapath Design Decisions

The following architectural decisions are frozen for Version 1 of the processor and shall not be modified without a formal revision:

Decision	Rationale
Harvard Architecture	Eliminates structural memory hazards
Separate Instruction/Data Memory	Allows concurrent IF and MEM access
Branch Resolution in EX Stage	Minimises branch penalty cycles
Static Not-Taken Prediction	Simple and sufficient for Version 1
2R1W Register File	Sufficient for single-issue in-order
EX/MEM and MEM/WB Forwarding	Eliminates most data hazards
Load-Use Hazard Stall	Correctly handles load-use dependency
Single-Cycle Memory Access	Simplifies pipeline control logic

8 Control Path Overview

The control path of the HK0411Bv1RV32 processor is responsible for generating all control signals required to coordinate instruction execution through the pipeline.

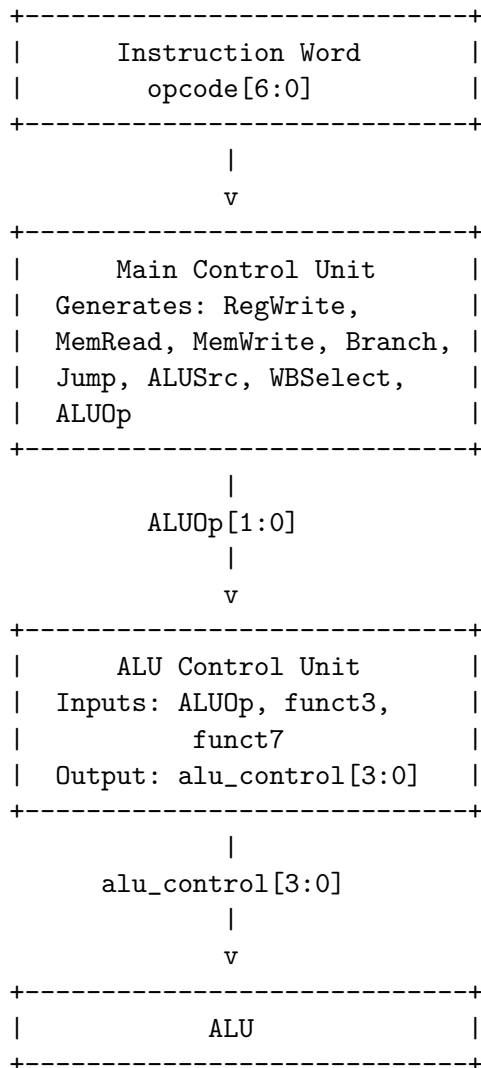
The control path interprets instruction fields, controls datapath multiplexers, initiates memory operations, manages register writeback, handles hazards, and controls branch recovery.

Control Path Major Blocks:

Block	Function
Main Control Unit	Decodes opcode and generates control signals
ALU Control Unit	Determines specific ALU operation
Branch Control Unit	Evaluates branch conditions
Forwarding Unit	Detects and resolves data dependencies
Hazard Detection Unit	Detects load-use hazards
Pipeline Flush Controller	Manages branch misprediction recovery

8.1 Control Path Organization

The overall control flow through the major control blocks is illustrated below.



The Main Control Unit generates high-level control signals while the ALU Control Unit generates the specific operation required by the Arithmetic Logic Unit.

8.2 Main Control Unit

The Main Control Unit decodes the instruction opcode field and generates the primary control signals used throughout the processor. It operates in the Instruction Decode (ID) stage.

Interface:

Signal	Direction	Description
opcode[6:0]	Input	Instruction opcode field
RegWrite	Output	Enable register file write
MemRead	Output	Enable data memory read
MemWrite	Output	Enable data memory write
Branch	Output	Indicates a branch instruction
Jump	Output	Indicates a jump instruction
ALUSrc	Output	Selects ALU operand B source
WBSelect	Output	Selects writeback data source
ALUOp	Output	ALU operation group encoding

Control Signal Truth Table:

Instruction	RegWrite	MemRead	MemWrite	Branch	Jump	ALUSrc	WBSelect
R-Type	1	0	0	0	0	0	ALU
I-Type	1	0	0	0	0	1	ALU
LW	1	1	0	0	0	1	MEM
SW	0	0	1	0	0	1	—
BEQ/BNE	0	0	0	1	0	0	—
JAL	1	0	0	0	1	1	PC+4
LUI	1	0	0	0	0	1	IMM

8.3 ALU Control Unit

The ALU Control Unit determines the exact ALU operation required for the current instruction based on the ALUOp field, funct3, and funct7. It operates in the Execute (EX) stage.

Interface:

Signal	Direction	Description
ALUOp	Input	High-level operation group
funct3	Input	Instruction funct3 field
funct7	Input	Instruction funct7 field
alu_control	Output	Specific ALU operation select

ALU Operation Encoding:

Operation	funct3	funct7	Description
ADD	000	0000000	Integer addition
SUB	000	0100000	Integer subtraction
SLL	001	0000000	Logical left shift
SLT	010	0000000	Signed set less than
SLTU	011	0000000	Unsigned set less than
XOR	100	0000000	Bitwise exclusive OR
SRL	101	0000000	Logical right shift
SRA	101	0100000	Arithmetic right shift
OR	110	0000000	Bitwise OR
AND	111	0000000	Bitwise AND

8.4 Branch Control Unit

The Branch Control Unit evaluates branch conditions in the Execute (EX) stage and determines whether control flow should be redirected.

Interface:

Signal	Direction	Description
rs1_data	Input	Value of source register 1
rs2_data	Input	Value of source register 2
branch_type	Input	Branch instruction type (funct3)
branch_taken	Output	Branch condition result

Supported Branch Conditions:

Instruction	funct3	Condition
BEQ	000	Branch if rs1 == rs2
BNE	001	Branch if rs1 != rs2

8.5 Forwarding Unit

The Forwarding Unit detects data dependencies between instructions in adjacent pipeline stages and selects the appropriate bypass paths to avoid unnecessary pipeline stalls. It operates in the Execute (EX) stage.

Interface:

Signal	Direction	Description
id_ex.rs1	Input	RS1 address in ID/EX register
id_ex.rs2	Input	RS2 address in ID/EX register
ex_mem.rd	Input	RD address in EX/MEM register
mem_wb.rd	Input	RD address in MEM/WB register
ex_mem.RegWrite	Input	RegWrite flag in EX/MEM register
mem_wb.RegWrite	Input	RegWrite flag in MEM/WB register
ForwardA	Output	Operand A forwarding select
ForwardB	Output	Operand B forwarding select

Forwarding Select Encoding:

Signal	Value	Source
ForwardA / ForwardB	00	Register File output
ForwardA / ForwardB	01	MEM/WB stage writeback value
ForwardA / ForwardB	10	EX/MEM stage ALU result

8.6 Hazard Detection Unit

The Hazard Detection Unit identifies load-use hazards that cannot be resolved through forwarding alone. It operates in the Instruction Decode (ID) stage and inserts a pipeline stall when required.

Interface:

Signal	Direction	Description
id_ex.MemRead	Input	Load instruction in EX stage
id_ex.rd	Input	Destination register of EX instruction
if_id.rs1	Input	RS1 of instruction in ID stage
if_id.rs2	Input	RS2 of instruction in ID stage
PCWrite	Output	Stall program counter when low
IF_ID_Write	Output	Stall IF/ID register when low
InsertBubble	Output	Insert NOP bubble into ID/EX register

Hazard Detection Condition:

```
if (id_ex.MemRead == 1) AND
    (id_ex.rd == if_id.rs1 OR
     id_ex.rd == if_id.rs2) THEN
    stall pipeline for one cycle
```

8.7 Pipeline Flush Controller

The Pipeline Flush Controller removes incorrectly fetched instructions from the pipeline after a taken branch or unconditional jump. It operates in the Execute (EX) stage.

Interface:

Signal	Direction	Description
branch_taken	Input	Branch condition result
jump	Input	Jump instruction detected
FlushIFID	Output	Flush the IF/ID pipeline register
FlushIDEX	Output	Flush the ID/EX pipeline register

Flush Activation Conditions:

Condition	Action
branch_taken == 1	Assert FlushIFID and FlushIDEX
jump == 1	Assert FlushIFID and FlushIDEX
Neither condition active	No flush, pipeline continues normally

8.8 Control Signal Summary

Signal	Generated By	Category
RegWrite	Main Control Unit	Register Control
WBSelect	Main Control Unit	Register Control
MemRead	Main Control Unit	Memory Control
MemWrite	Main Control Unit	Memory Control
ALUSrc	Main Control Unit	ALU Control
ALUOp	Main Control Unit	ALU Control
alu_control	ALU Control Unit	ALU Control
Branch	Main Control Unit	Branch Control
branch_taken	Branch Control Unit	Branch Control
Jump	Main Control Unit	Branch Control
ForwardA	Forwarding Unit	Hazard Management
ForwardB	Forwarding Unit	Hazard Management
PCWrite	Hazard Detection Unit	Pipeline Control
IF_ID_Write	Hazard Detection Unit	Pipeline Control
InsertBubble	Hazard Detection Unit	Pipeline Control
FlushIFID	Pipeline Flush Controller	Pipeline Control
FlushIDEX	Pipeline Flush Controller	Pipeline Control

9 Immediate Generator Specification

The Immediate Generator (IMM GEN) is responsible for extracting and reconstructing immediate operands from RISC-V instruction encodings. The generated immediate value is provided to the Execute stage for arithmetic operations, address calculations, branch target generation, and jump target generation. The Immediate Generator operates combinationally during the Instruction Decode (ID) stage.

9.1 Supported Immediate Types

The Immediate Generator shall support the following immediate formats, all producing 32-bit sign-extended or zero-padded results:

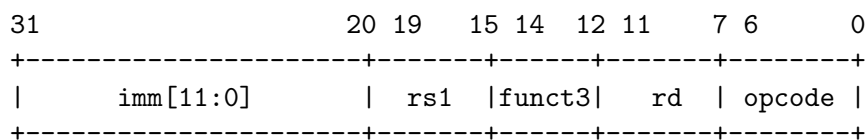
Immediate Type	Description	Used By
I-Type	12-bit sign-extended	ALU immediates, Loads
S-Type	12-bit sign-extended	Store instructions
B-Type	13-bit sign-extended, LSB=0	Conditional branches
U-Type	20-bit upper immediate	LUI
J-Type	21-bit sign-extended, LSB=0	Unconditional jumps

9.2 I-Type Immediate

Applicable Instructions:

Instruction	Description	Operation
ADDI	Add Immediate	ADD with immediate
SLTI	Set Less Than Immediate	Signed comparison
SLTIU	Set Less Than Immediate Unsigned	Unsigned comparison
XORI	XOR Immediate	Bitwise XOR
ORI	OR Immediate	Bitwise OR
ANDI	AND Immediate	Bitwise AND
SLLI	Shift Left Logical Immediate	Logical left shift
SRLI	Shift Right Logical Immediate	Logical right shift
SRAI	Shift Right Arithmetic Immediate	Arithmetic right shift
LW	Load Word	Memory address calculation

Instruction Format:



Immediate Reconstruction:

```
imm = {{20{instr[31]}}, instr[31:20]};
```

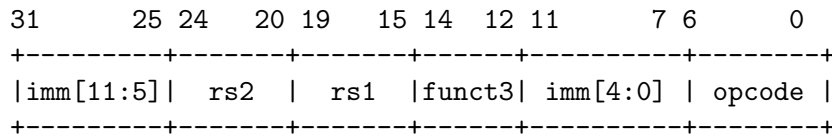
The immediate is sign-extended from bit 11 to produce a 32-bit value.

9.3 S-Type Immediate

Applicable Instructions:

Instruction	Description
SW	Store Word

Instruction Format:



Immediate Reconstruction:

```
imm = {{20{instr[31]}}, instr[31:25], instr[11:7]};
```

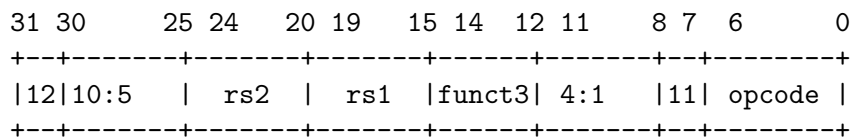
The immediate is assembled from two non-contiguous fields and sign-extended to 32 bits.

9.4 B-Type Immediate

Applicable Instructions:

Instruction	Description
BEQ	Branch if Equal
BNE	Branch if Not Equal

Instruction Format:



Immediate Reconstruction:

```
imm = {{19{instr[31]}}, instr[31], instr[7], instr[30:25], instr[11:8], 1'b0};
```

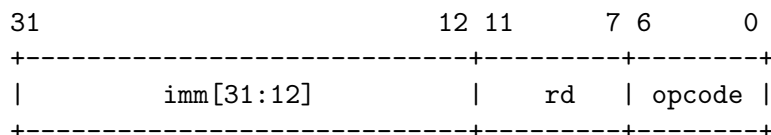
The least significant bit is implicitly zero, ensuring branch targets are aligned to two-byte boundaries. The immediate is sign-extended to 32 bits.

9.5 U-Type Immediate

Applicable Instructions:

Instruction	Description
LUI	Load Upper Immediate

Instruction Format:



Immediate Reconstruction:

```
imm = {instr[31:12], 12'b0};
```

The immediate occupies the upper 20 bits of the 32-bit result, with the lower 12 bits set to zero. No sign extension is performed.

9.6 J-Type Immediate

Applicable Instructions:

Instruction	Description
JAL	Jump and Link

Instruction Format:

31	30		21	20	19		12	11		7	6		0
+---+-----+---+-----+---+-----+---+-----+													
20		10:1		11		19:12				rd		opcode	
+---+-----+---+-----+---+-----+---+-----+													

Immediate Reconstruction:

```
imm = {{11{instr[31]}}, instr[31], instr[19:12], instr[20], instr[30:21], 1'b0};
```

The least significant bit is implicitly zero, ensuring jump targets are aligned to two-byte boundaries. The immediate is sign-extended to 32 bits.

9.7 Immediate Type Selection

The Immediate Generator shall determine the immediate format based on the instruction opcode as decoded by the Main Control Unit.

Instruction Category	Opcode	Immediate Format
I-Type Arithmetic	0010011	I-Type
Load Instructions	0000011	I-Type
Store Instructions	0100011	S-Type
Branch Instructions	1100011	B-Type
LUI	0110111	U-Type
JAL	1101111	J-Type

9.8 Module Interface

Signal	Width (bits)	Direction	Description
instr	32	Input	Raw instruction word
imm	32	Output	Generated immediate value

9.9 Design Requirements

Requirement	Description
Combinational Operation	Immediate valid within same clock cycle
Sign Extension	I, S, B, J types sign-extended to 32 bits
Zero Padding	U-type zero-padded in lower 12 bits
Alignment Enforcement	B and J types force LSB to zero
All Supported Formats	Must generate all five immediate types

10 Control Signal Specification

The HK0411Bv1RV32 processor utilizes a collection of control signals to coordinate datapath operation, memory accesses, register writeback, branch handling, and pipeline control.

Control signals are generated in the Instruction Decode (ID) stage by the Main Control Unit and propagated through the pipeline registers to the stages where they are required.

10.1 Control Signal Summary

Signal	Width (bits)	Description	Generated In
RegWrite	1	Register file write enable	ID Stage
MemRead	1	Data memory read enable	ID Stage
MemWrite	1	Data memory write enable	ID Stage
Branch	1	Branch instruction indicator	ID Stage
Jump	1	Jump instruction indicator	ID Stage
ALUSrc	1	ALU operand B source select	ID Stage
WBSelect	2	Writeback source select	ID Stage
ALUOp	3	ALU operation category encoding	ID Stage

10.2 RegWrite

RegWrite controls whether the result of an instruction is written back to the destination register in the Register File.

RegWrite	Behaviour	Applicable Instructions
1	Write to destination register	R-Type, I-Type, LW, JAL, Lui
0	No register write	SW, BEQ, BNE

10.3 MemRead

MemRead enables a read operation on the Data Memory in the Memory Access stage.

MemRead	Behaviour	Applicable Instructions
1	Read from data memory	LW
0	No data memory read	All other instructions

10.4 MemWrite

MemWrite enables a write operation on the Data Memory in the Memory Access stage.

MemWrite	Behaviour	Applicable Instructions
1	Write to data memory	SW
0	No data memory write	All other instructions

10.5 Branch

Branch identifies whether the current instruction is a conditional branch instruction. This signal is used by the Branch Control Unit in the Execute stage to enable branch condition evaluation.

Branch	Behaviour	Applicable Instructions
1	Branch condition evaluated	BEQ, BNE
0	Not a branch instruction	All other instructions

10.6 Jump

Jump identifies whether the current instruction is an unconditional jump instruction. When asserted, the pipeline is flushed and the Program Counter is redirected to the jump target address.

Jump	Behaviour	Applicable Instructions
1	Redirect PC to jump target	JAL
0	Not a jump instruction	All other instructions

10.7 ALUSrc

ALUSrc selects the source of ALU Operand B. When deasserted, the register file output is used. When asserted, the immediate generator output is used.

ALUSrc	Operand B Source	Applicable Instructions
0	Register file output (rs2_data)	R-Type, BEQ, BNE
1	Immediate generator output	I-Type, LW, SW, JAL, LUI

10.8 WBSelect

WBSelect determines which data source is written back to the destination register in the Write Back stage.

WBSelect	Writeback Source	Applicable Instructions
00	ALU result	R-Type, I-Type arithmetic
01	Data memory read data	LW
10	PC + 4	JAL
11	Immediate value	LUI

10.9 ALUOp

ALUOp specifies the category of ALU operation required. The ALU Control Unit combines ALUOp with the instruction funct3 and funct7 fields to generate the final ALU operation control code.

ALUOp	Operation Category	Applicable Instructions
000	Load and store address calculation	LW, SW
001	Branch comparison	BEQ, BNE
010	R-Type arithmetic	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
011	I-Type arithmetic	ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI
100	Upper immediate operation	LUI
101	Jump operation	JAL
110	Reserved	—
111	Reserved	—

10.10 Instruction Control Signal Truth Table

The complete control signal assignments for all supported instructions are defined in the table below.

Instruction	RegWrite	MemRead	MemWrite	Branch	Jump	ALUSrc	WBSelect	ALUOp
ADD	1	0	0	0	0	0	00	010
SUB	1	0	0	0	0	0	00	010
SLL	1	0	0	0	0	0	00	010
SLT	1	0	0	0	0	0	00	010
SLTU	1	0	0	0	0	0	00	010
XOR	1	0	0	0	0	0	00	010
SRL	1	0	0	0	0	0	00	010
SRA	1	0	0	0	0	0	00	010
OR	1	0	0	0	0	0	00	010
AND	1	0	0	0	0	0	00	010
ADDI	1	0	0	0	0	1	00	011
SLTI	1	0	0	0	0	1	00	011
SLTIU	1	0	0	0	0	1	00	011
XORI	1	0	0	0	0	1	00	011
ORI	1	0	0	0	0	1	00	011
ANDI	1	0	0	0	0	1	00	011
SLLI	1	0	0	0	0	1	00	011
SRLI	1	0	0	0	0	1	00	011
SRAI	1	0	0	0	0	1	00	011
LW	1	1	0	0	0	1	01	000
SW	0	0	1	0	0	1	XX	000
BEQ	0	0	0	1	0	0	XX	001
BNE	0	0	0	1	0	0	XX	001
JAL	1	0	0	0	1	1	10	101
LUI	1	0	0	0	0	1	11	100

11 ALU Operation Encoding

The HK0411Bv1RV32 processor utilizes an internal ALU control bus to select the operation performed by the Arithmetic Logic Unit (ALU).

The ALU Control Unit generates the ALU control code based on the following inputs:

Input	Description
ALUOp	High-level operation category from Main Control Unit
funct3	Instruction funct3 field
funct7	Instruction funct7 field

The generated ALU control code is supplied to the ALU during the Execute (EX) stage.

11.1 ALU Control Signal Specification

Signal	Width (bits)	Description
alu_control	4	Selects the ALU operation to be performed
zero	1	Asserted when the ALU result equals zero

11.2 ALU Control Encoding

alu_control	Operation	Description
0000	ADD	Integer addition
0001	SUB	Integer subtraction
0010	SLL	Logical left shift
0011	SLT	Signed set less than comparison
0100	SLTU	Unsigned set less than comparison
0101	XOR	Bitwise exclusive OR
0110	SRL	Logical right shift
0111	SRA	Arithmetic right shift
1000	OR	Bitwise OR
1001	AND	Bitwise AND
1010	PASS IMM	Pass immediate value directly to output
1011	Reserved	—
1100	Reserved	—
1101	Reserved	—
1110	Reserved	—
1111	Reserved	—

11.3 Supported ALU Operations

Category	Operation	Description
Arithmetic	ADD	Integer addition
	SUB	Integer subtraction
Logical	AND	Bitwise AND
	OR	Bitwise OR
	XOR	Bitwise exclusive OR
Shift	SLL	Logical left shift
	SRL	Logical right shift
	SRA	Arithmetic right shift
Comparison	SLT	Signed set less than
	SLTU	Unsigned set less than
Pass Through	PASS IMM	Forward immediate to output

11.4 ALU Control Generation Rules

The ALU Control Unit shall decode the instruction type according to the rules defined in the following subsections.

11.4.1 Load and Store Instructions

Instruction	ALU Operation	alu_control
LW	ADD	0000
SW	ADD	0000

The ALU computes the effective memory address as follows:

`effective_address = base_register + sign_extended_immediate`

11.4.2 Branch Instructions

Instruction	ALU Operation	alu_control
BEQ	SUB	0001
BNE	SUB	0001

The subtraction result and the zero flag are used by the Branch Control Unit to determine whether the branch condition is satisfied.

11.4.3 Jump Instructions

Instruction	ALU Operation	alu_control
JAL	ADD	0000

The ALU computes the jump target address. The return address $PC + 4$ is passed to the writeback stage through a separate datapath.

11.4.4 Upper Immediate Instructions

Instruction	ALU Operation	alu_control
LUI	PASS IMM	1010

The ALU shall forward the upper immediate value directly to the writeback path without performing any arithmetic operation.

11.5 Complete Instruction-to-ALU Control Mapping

Instruction	ALU Operation	alu_control
ADD	ADD	0000
SUB	SUB	0001
SLL	SLL	0010
SLT	SLT	0011
SLTU	SLTU	0100
XOR	XOR	0101
SRL	SRL	0110
SRA	SRA	0111
OR	OR	1000
AND	AND	1001
ADDI	ADD	0000
SLTI	SLT	0011
SLTIU	SLTU	0100
XORI	XOR	0101
ORI	OR	1000
ANDI	AND	1001
SLLI	SLL	0010
SRLI	SRL	0110
SRAI	SRA	0111
LW	ADD	0000
SW	ADD	0000
BEQ	SUB	0001
BNE	SUB	0001
JAL	ADD	0000
LUI	PASS IMM	1010

11.6 ALU Zero Flag

The ALU shall generate a zero status flag used by the Branch Control Unit during branch condition evaluation.

Signal	Width (bits)	Description
zero	1	Asserted when the ALU result equals zero

Zero Flag Usage:

Instruction	Condition	Branch Taken When
BEQ	rs1 == rs2	zero == 1
BNE	rs1 != rs2	zero == 0

12 Pipeline Register Specification

Pipeline registers isolate adjacent pipeline stages and preserve instruction state as instructions progress through the processor pipeline.

The HK0411Bv1RV32 processor implements four pipeline registers:

Pipeline Register	Purpose
IF/ID	Transfer fetched instruction to Decode
ID/EX	Transfer decoded information to Execute
EX/MEM	Transfer execution results to Memory
MEM/WB	Transfer final results to Writeback

All pipeline registers shall support:

Feature	Description
Clocked Update	Rising edge triggered
Synchronous Reset	Active-high synchronous reset
Stall Operation	Hold contents when stall asserted
Flush Operation	Clear contents when flush asserted

12.1 IF/ID Pipeline Register

The IF/ID pipeline register transfers instruction fetch information from the Instruction Fetch (IF) stage to the Instruction Decode (ID) stage.

12.1.1 Stored Signals

Signal	Width (bits)	Description
if_id_pc	32	Current program counter value
if_id_pc_plus4	32	Sequential next instruction address
if_id_instr	32	Fetched 32-bit instruction word
if_id_valid	1	Instruction valid indicator

12.1.2 Functional Requirements

Condition	Behaviour
Normal operation	Capture instruction fetch results each cycle
Stall asserted	Hold all register contents unchanged
Flush asserted	Clear all register contents to zero

12.2 ID/EX Pipeline Register

The ID/EX pipeline register transfers decoded instruction information, operands, immediate values, and control signals from the Instruction Decode (ID) stage to the Execute (EX) stage.

12.2.1 Datapath Signals

Signal	Width (bits)	Description
id_ex_pc	32	Current program counter value
id_ex_pc_plus4	32	Sequential next instruction address
id_ex_rs1_data	32	Source operand 1 register value
id_ex_rs2_data	32	Source operand 2 register value
id_ex_imm	32	Sign-extended immediate value
id_ex_rs1	5	Source register 1 address
id_ex_rs2	5	Source register 2 address
id_ex_rd	5	Destination register address
id_ex_funct3	3	Instruction funct3 field
id_ex_funct7	7	Instruction funct7 field

12.2.2 Control Signals

Signal	Width (bits)	Description
RegWrite	1	Enable register file write port
MemRead	1	Enable data memory read operation
MemWrite	1	Enable data memory write operation
Branch	1	Indicates a branch instruction
Jump	1	Indicates a jump instruction
ALUSrc	1	Select immediate as ALU operand B
WBSelect	2	Select writeback data source
ALUOp	3	ALU operation category encoding

12.2.3 Functional Requirements

Condition	Behaviour
Normal operation	Transfer all decoded signals to Execute stage
Bubble insertion	Clear all control signals to zero
Flush asserted	Clear all register contents to zero

12.3 EX/MEM Pipeline Register

The EX/MEM pipeline register transfers execution results from the Execute (EX) stage to the Memory Access (MEM) stage.

12.3.1 Datapath Signals

Signal	Width (bits)	Description
ex_mem_alu_result	32	ALU computation result
ex_mem_store_data	32	Data to be written to memory
ex_mem_branch_target	32	Computed branch target address
ex_mem_pc_plus4	32	Sequential next instruction address
ex_mem_imm	32	Immediate value for LUI writeback
ex_mem_rd	5	Destination register address

12.3.2 Control Signals

Signal	Width (bits)	Description
RegWrite	1	Enable register file write port
MemRead	1	Enable data memory read operation
MemWrite	1	Enable data memory write operation
WBSelect	2	Select writeback data source

12.3.3 Functional Requirements

Condition	Behaviour
Normal operation	Transfer all execution results to Memory stage
Flush asserted	Clear all register contents to zero

12.4 MEM/WB Pipeline Register

The MEM/WB pipeline register transfers memory access and execution results from the Memory Access (MEM) stage to the Write Back (WB) stage.

12.4.1 Datapath Signals

Signal	Width (bits)	Description
mem_wb_mem_data	32	Data word read from data memory
mem_wb_alu_result	32	ALU computation result
mem_wb_pc_plus4	32	Sequential next instruction address
mem_wb_imm	32	Immediate value for LUI writeback
mem_wb_rd	5	Destination register address

12.4.2 Control Signals

Signal	Width (bits)	Description
RegWrite	1	Enable register file write port
WBSelect	2	Select writeback data source

12.4.3 Functional Requirements

Condition	Behaviour
Normal operation	Transfer all results to Writeback stage

12.5 Writeback Source Encoding

The writeback multiplexer shall select one of four possible writeback sources according to the WBSelect control signal encoding defined below.

WBSelect	Writeback Source	Applicable Instructions
00	ALU Result	R-Type, I-Type arithmetic
01	Data Memory Read Data	Load instructions (LW)
10	PC + 4 Value	Jump and link (JAL)
11	Immediate Value	Load upper immediate (LUI)

12.6 Pipeline Register Summary

Register	Stall Support	Flush Support	Purpose
IF/ID	Yes	Yes	Transfer fetched instruction to Decode stage
ID/EX	No	Yes	Transfer decoded information to Execute stage
EX/MEM	No	No	Transfer execution results to Memory stage
MEM/WB	No	No	Transfer final results to Writeback stage

13 Hazard Management Overview

Pipeline hazards occur when the normal flow of instruction execution cannot be maintained. The HK0411Bv1RV32 processor implements mechanisms to detect and recover from data hazards and control hazards. Structural hazards are eliminated through the use of a Harvard Architecture with separate instruction and data memories.

13.1 Structural Hazards

The processor shall not experience structural hazards during normal operation. Instruction fetches and data memory accesses are performed using entirely separate memory interfaces.

Resource	Implementation
Instruction Memory	Dedicated read-only interface
Data Memory	Dedicated read/write interface
Access Paths	Independent, no shared bus

13.2 Data Hazards

Data hazards occur when an instruction requires an operand that has not yet been written back by an earlier instruction in the pipeline.

Example:

```
ADD x1, x2, x3    # Produces x1
SUB x4, x1, x5    # Consumes x1 before writeback completes
```

The processor resolves data hazards through a dedicated forwarding mechanism that bypasses register writeback where possible.

13.3 Forwarding Mechanism

The Forwarding Unit detects producer-consumer dependencies between instructions in adjacent pipeline stages and supplies operand values directly to the Execute stage without waiting for register writeback.

Forwarding Path	Source Register	Latency Saved
EX/MEM → EX	ex_mem.rd	1 cycle
MEM/WB → EX	mem_wb.rd	2 cycles

Forwarding Operand Select Encoding:

ForwardA/B	Operand Source	Condition
00	Register File output	No hazard detected
01	MEM/WB writeback value	MEM/WB hazard detected
10	EX/MEM ALU result	EX/MEM hazard detected

13.4 Load-Use Hazards

A load-use hazard occurs when an instruction immediately following a load instruction requires the loaded value. Forwarding alone cannot resolve this dependency because the loaded data is not available until the end of the Memory Access stage.

Example:

```
LW  x1, 0(x2)    # Load produces x1 at end of MEM stage
ADD x3, x1, x4    # Requires x1 at start of EX stage
```

Load-Use Hazard Detection Condition:

```
if (id_ex.MemRead == 1) AND
   (id_ex.rd == if_id.rs1 OR
    id_ex.rd == if_id.rs2) THEN
    stall pipeline for one cycle
```

Recovery Actions:

Action	Description
Stall Program Counter	PCWrite deasserted for one cycle
Stall IF/ID Register	IF_ID.Write deasserted for one cycle
Insert Bubble	ID/EX control signals cleared to zero

13.5 Control Hazards

Control hazards occur when the next Program Counter value depends on the outcome of a branch instruction that has not yet been resolved.

Supported Branch Instructions:

Instruction	Condition	Resolution Stage
BEQ	Branch if rs1 == rs2	Execute (EX)
BNE	Branch if rs1 != rs2	Execute (EX)

Control hazards shall be resolved using static not-taken prediction combined with pipeline flushing and Program Counter redirection when a branch is determined to be taken.

13.6 Hazard Management Summary

Hazard Type	Resolution Mechanism	Pipeline Penalty
Structural Hazard	Eliminated by Harvard Architecture	0 cycles
Data Hazard	Forwarding Unit	0 cycles
Load-Use Hazard	Stall and bubble insertion	1 cycle
Control Hazard (Taken)	Flush and PC redirection	2 cycles
Control Hazard (NT)	No action required	0 cycles

14 Branch Prediction Strategy

The HK0411Bv1RV32 processor implements a static branch prediction policy. Branch instructions are predicted as not taken and the pipeline continues fetching sequential instructions until the branch outcome is resolved in the Execute stage.

14.1 Prediction Policy

Parameter	Value
Prediction Strategy	Static Not-Taken
Resolution Stage	Execute (EX)
Predicted Next PC	PC + 4
Branch Penalty (Taken)	2 cycles
Branch Penalty (Not Taken)	0 cycles

Predicted Next PC = PC + 4

14.2 Branch Resolution

Branch instructions are resolved in the Execute (EX) stage. The Branch Comparator evaluates the branch condition and the Branch Target Adder computes the target address simultaneously.

Operation	Description
Condition Evaluation	Compare rs1 and rs2 per branch type
Target Computation	branch_target = PC + branch_imm
Decision Output	Assert branch_taken if condition is true

14.3 Misprediction Recovery

When a branch is determined to be taken, the two incorrectly fetched instructions must be removed from the pipeline and execution must resume from the branch target address.

Recovery Sequence:

Step	Action	Affected Register
1	Flush IF/ID register	IF/ID cleared to NOP
2	Flush ID/EX register	ID/EX cleared to NOP
3	Redirect Program Counter	PC $\leftarrow$ branch_target

14.4 Future Enhancements

The following branch prediction enhancements may be considered for future processor versions:

Enhancement	Description
Static Backward-Taken	Predict taken for negative offsets
Dynamic Branch Prediction	Runtime history-based prediction
Branch History Table	Per-branch saturating counter table
Branch Target Buffer	Cache of predicted target addresses

15 Memory Architecture

The HK0411Bv1RV32 processor employs a Harvard Architecture consisting of independent instruction and data memory systems. This eliminates structural hazards between simultaneous instruction fetch and data memory access operations.

15.1 Instruction Memory

Instruction Memory stores the executable program and is accessed exclusively by the Instruction Fetch stage.

Parameter	Value
Access Type	Read-only during processor execution
Instruction Width	32 bits
Access Latency	Single-cycle
Interface	Independent from Data Memory

Memory Interface:

Signal	Direction	Description
pc_address	Input	Program counter address
instruction	Output	32-bit fetched instruction word

15.2 Data Memory

Data Memory stores program data and is accessed exclusively by the Memory Access stage.

Parameter	Value
Access Type	Read and write
Data Width	32 bits
Access Latency	Single-cycle
Interface	Independent from Instruction Memory

Memory Interface:

Signal	Direction	Description
mem_address	Input	Effective memory address
mem_write_data	Input	Data word to be stored
mem_read	Input	Memory read enable signal
mem_write	Input	Memory write enable signal
mem_read_data	Output	Data word read from memory

Supported Operations:

Instruction	Operation	Description
LW	Read	Load 32-bit word from memory
SW	Write	Store 32-bit word to memory

15.3 Address Space

Parameter	Value
Address Width	32 bits
Theoretical Address Space	4 GB
Alignment Requirement	Word-aligned (4-byte boundary)

The actual implemented memory size may be smaller than the full 32-bit theoretical address space.

15.4 Memory Alignment

The processor assumes word-aligned accesses for all supported memory operations. Misaligned memory accesses are not supported in Version 1.

Instruction	Alignment Requirement	Access Width
LW	4-byte word aligned	32 bits
SW	4-byte word aligned	32 bits

Part III

IMPLEMENTATION AND VERIFICATION SPECIFICATION

16 RTL Module Specification

16.1 pc.sv

16.2 instr_mem.sv

16.3 data_mem.sv

16.4 regfile.sv

16.5 imm_gen.sv

16.6 alu.sv

16.7 alu_control.sv

16.8 control_unit.sv

16.9 forwarding_unit.sv

16.10 hazard_unit.sv

16.11 if_id.sv

16.12 id_ex.sv

16.13 ex_mem.sv

16.14 mem_wb.sv

16.15 cpu_top.sv

17 Verification Requirements

The processor shall be verified using a self-checking SystemVerilog-based verification environment. Verification activities shall demonstrate functional correctness of all supported instructions and hazard handling mechanisms before the design is considered complete.

17.1 Verification Environment

Component	Function
Generator	Produces instruction stimulus sequences
Driver	Applies stimulus to the processor interface
Monitor	Observes and captures processor outputs
Scoreboard	Compares actual outputs against expected
Coverage Collector	Measures functional coverage progress

17.2 Directed Testing

Test Category	Instructions Covered
Arithmetic Instructions	ADD, SUB, ADDI
Logical Instructions	AND, OR, XOR, ANDI, ORI, XORI
Shift Instructions	SLL, SRL, SRA, SLLI, SRLI, SRAI
Comparison Instructions	SLT, SLTU, SLTI, SLTIU
Load Instructions	LW
Store Instructions	SW
Branch Instructions	BEQ, BNE
Jump Instructions	JAL
Upper Immediate	LUI

17.3 Hazard Verification

Hazard Scenario	Expected Behaviour
EX/MEM Forwarding	Correct operand supplied in EX stage
MEM/WB Forwarding	Correct operand supplied in EX stage
Load-Use Hazard Recovery	One stall cycle inserted correctly
Branch Taken Flush	IF/ID and ID/EX flushed correctly
Branch Not-Taken	Pipeline continues without flush

17.4 Functional Coverage

Coverage Point	Description
All Supported Instructions	Every instruction executed at least once
All ALU Operations	All ten ALU operations exercised
Branch Taken	BEQ and BNE taken cases covered
Branch Not-Taken	BEQ and BNE not-taken cases covered
EX/MEM Forwarding Path	Forward from EX/MEM register exercised
MEM/WB Forwarding Path	Forward from MEM/WB register exercised
Load-Use Hazard Events	At least one load-use stall observed
Pipeline Flush Events	At least one branch flush observed

Target Functional Coverage Goal: $\geq 95\%$

17.5 Assertion-Based Verification

SystemVerilog Assertions shall be used to verify critical architectural invariants throughout simulation.

Assertion	Property Verified
x0 Zero Invariant	Register x0 always reads as zero
No Write to x0	Register x0 is never written
Flush Clears Pipeline	Flush zeroes all control signals
Stall Preserves Contents	Stalled registers hold their values
Valid Forward Select	ForwardA and ForwardB are always valid

17.6 Verification Completion Criteria

Verification shall be considered complete when all of the following criteria have been satisfied:

Item	Criterion	Target
1	All directed tests pass	100% pass rate
2	All hazard tests pass	100% pass rate
3	Functional coverage goal achieved	$\geq 95\%$
4	All assertions pass	Zero assertion failures
5	No unresolved simulation failures	Zero open failures

18 Verification Plan

18.1 Instruction Coverage Matrix

18.2 Hazard Coverage Matrix

18.3 Functional Coverage Plan

18.4 Assertion Plan